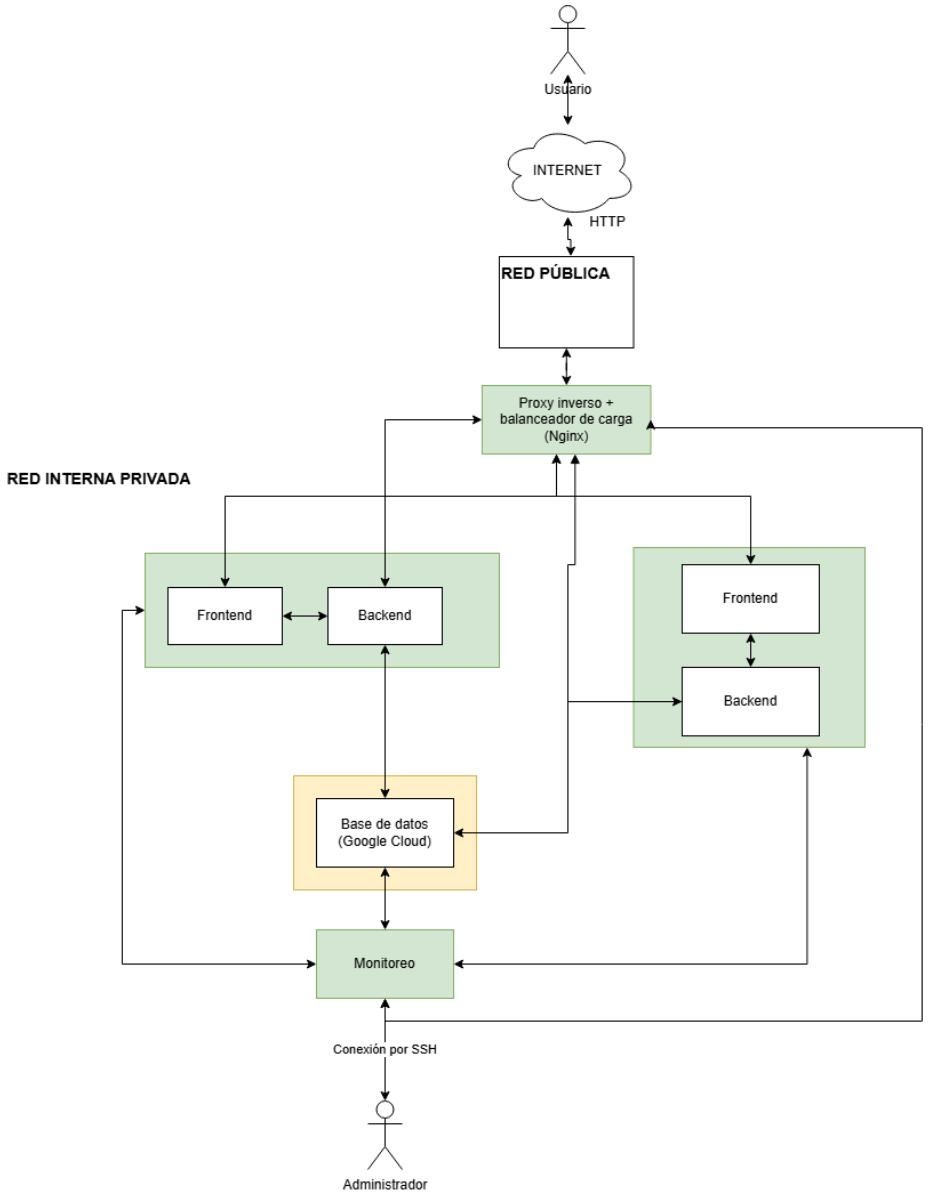
Proyecto Unidad 1 Administración de Sistemas Computacionales:
Sistema audio visual para CORES

Integrantes: Carlos Hormazabal, Vicente Farías

Introducción

Este documento detalla la arquitectura y las configuraciones de red que implementamos para el despliegue del "Sistema audiovisual para CORES". El objetivo del proyecto es entregar una plataforma dinámica y segura, capaz de gestionar eventos en tiempo real como votaciones, control de asistencia y asignación de turnos de palabra. Para garantizar que el sistema esté siempre disponible y protegido, diseñamos una infraestructura en Google Cloud que se apoya fuertemente en la segmentación de redes, reglas de firewall bien definidas y un sistema de respaldos automatizados.

Diagrama Arquitectura:



1. Distribución de la Red y Arquitectura

La infraestructura se ha segmentado estratégicamente en dos redes, pública y privada, con el objetivo de garantizar la máxima seguridad. Se tomó la decisión de aislar en capas, ya que si todos los servicios y bases de datos estuvieran expuestos directamente a Internet, la probabilidad de sufrir un ataque cibernético sería sumamente alta. Al establecer esta división, se crea una barrera que aísla el sistema, protegiendo la lógica de negocio y los datos frente a vulnerabilidades externas.

En base a esto, la red se divide de la siguiente manera:

- **Red Pública:** Funciona como la "puerta de enlace" y el único punto de entrada para los usuarios que acceden desde Internet mediante el protocolo HTTP. En esta zona funciona exclusivamente el proxy inverso y balanceador de carga (Nginx), cuya única función es recibir las peticiones externas y enviarlas de forma segura hacia el interior.
- **Red Interna Privada:** Es un entorno completamente invisible e inaccesible desde Internet, diseñado para alojar todos los servicios críticos. Aquí operan los servidores de Frontend y Backend y la Base de Datos, comunicándose entre sí de forma protegida.

2. Políticas de Seguridad y Reglas de Firewall

Para permitir el puerto 3000 en el firewall se le da uso al comando “sudo ufw allow 3000/tcp”, El comando añade una regla al firewall UFW para permitir conexiones entrantes por el puerto 3000 usando el protocolo TCP, ejecutándose con privilegios de administrador mediante sudo.

Para asegurar el control del flujo de datos y proteger los recursos de la infraestructura, se diseñó un conjunto de políticas de firewall personalizadas, segmentadas estrictamente entre la red pública y la red privada. Esto permite un control del tráfico, estructurado de la siguiente manera:

Tráfico Público y de Entrada (Red Pública):

- **public-network-allow-http (Puerto 80):** Permite el tráfico web que entra HTTP, hacia las instancias de la red pública.Esta regla es vital para que los usuarios puedan acceder al sistema interactuando con el proxy inverso Nginx desde Internet.
- **allow-web-ssh (Puertos 22 y 80):** Habilita la administración remota por consola (SSH) hacia los servidores de la zona pública. Facilita a los administradores la configuración, el mantenimiento y el despliegue del proxy inverso.

Tráfico Interno y de Aplicación (Red Privada):

- **allow-db-interna (Puerto 5432):** Habilita el tráfico hacia el puerto de PostgreSQL desde la red privada. Esta decisión es fundamental, garantiza que la capa de datos este aislada, inaccesible y segura ante cualquier intento de conexión directa desde el exterior.
- **allow-puerto-3000-5173 (Puertos 3000 y 5173):** Permite la comunicación hacia los puertos 3000 (Backend en NestJS/Node) y 5173 (Frontend en React) únicamente a nivel de red interna. Restringir estos puertos asegura que el tráfico fluya obligatoriamente a través del balanceador de carga público, evitando que usuarios de Internet accedan a los microservicios de forma directa.
- **allow-ssh-private-net (Puerto 22):** Permite conexiones SSH exclusivamente a la red privada. Esta regla es la base para la implementación del modelo de Bastion Host, obliga a que cualquier trabajo de administración sobre los servidores críticos se realice conectándose primero al servidor server-monitor, reduciendo la exposición de ataques cibernéticos.

3. Capa de Datos

Se optó por utilizar Google Cloud SQL, un servicio de base de datos relacional totalmente gestionado. La justificación principal de esta decisión es la alta disponibilidad nativa que provee la plataforma. Alcanzar un nivel similar de resiliencia mediante una base de datos autogestionada representaría un proceso manual complejo y un aumento directo en los costos, ya que exigiría la configuración y mantenimiento de al menos dos máquinas virtuales adicionales para asegurar una alta disponibilidad en el sistema. Por motivos de seguridad, la instancia de Cloud SQL es accesible de forma exclusiva desde la red interna privada. Esto garantiza que únicamente las maquinas que almacenan el Backend, principal y réplica, puedan realizar operaciones de lectura y escritura, manteniendo los datos completamente aislados de cualquier acceso no autorizado desde Internet.

Archivos multimedia:

En el sistema existe un directorio especial denominado “uploads”, el cual contiene todos los archivos multimedia, tales como videos, imágenes y archivos PDF. Esto ocasiona problemas de sincronización, puesto que existen dos máquinas virtuales que cuentan con su propio directorio “uploads”; por consiguiente, al cargarse un archivo, inevitablemente uno de los dos directorios queda desactualizado.

4. Estrategia de Respaldos (Backups)

Para asegurar la disponibilidad de la información, se implementó la Regla 3-2-1:

- **3 Copias de los datos:** Se mantiene la base de datos viva en producción, es decir, Cloud SQL, respaldos internos que son generados por el servidor y exportaciones en formato .sql.gz.
- **2 Soportes de almacenamiento:** Los datos transaccionales residen en discos SSD de Cloud SQL, mientras que los archivos de respaldo se almacenan en Buckets de Google Cloud Storage.
- **1 Copia Off-site:** Las exportaciones se envían a un bucket deslocalizado (gs://backups-gobierno-regional-cl), funcionando como una bóveda externa. Si la red de producción se viera comprometida, estos datos permanecen intactos y accesibles.

Automatización y Privilegios: El proceso se ejecuta a través de un script Bash (backup_gcp.sh) alojado en el servidor server-monitor, el cual delega la carga de la exportación a la API de Google mediante el parámetro --offload. Para evitar errores de autorización, HTTP 403, se aplicó el principio de Mínimo Privilegio configurando una cuenta de servicio en IAM con el rol estricto de Administrador de Cloud SQL. La tarea se automatizó mediante el planificador crontab de Linux para ejecutarse todos los días a las 03:00 AM, derivando los logs a un archivo de auditoría local (backup_cron.log).

5. Administración y Monitoreo

Para la administración de la infraestructura, se implementó una máquina virtual centralizada que actúa como nodo de acceso seguro llamada server-monitor. La gestión operativa de los componentes, Proxy, Frontends, Backends y Base de Datos, se realiza conectándose a este servidor central vía SSH. Esta arquitectura fortalece la seguridad, ya que centraliza el control y evita que el administrador deba abrir conexiones públicas directas hacia cada servidor individual. Por otro lado, el monitoreo continuo del rendimiento, consumo de recursos y la salud general de la infraestructura se delega a las herramientas de observabilidad de Google Cloud (Cloud Monitoring), lo que permite visualizar métricas precisas y automatizadas sin generar sobrecarga de procesamiento en las máquinas virtuales.

6. Tecnologías Utilizadas y Justificación

- **Google Cloud:** Proveedor seleccionado para alojar la infraestructura por su oferta de recursos escalables y servicios administrados, eliminando los cuellos de botella y costos de mantener hardware físico.
- **Nginx (Balanceador y Proxy):** La elección frente a soluciones como Google Cloud Load Balancer es debido a tres factores importantes. En primer lugar, por su rentabilidad al ser una herramienta gratuita. En segundo lugar, por su versatilidad, ya que permite aprovechar un mismo servicio para cumplir la doble función de proxy inverso y balanceador de carga. Finalmente se decidió por Nginx, porque ya se contaba con experiencia previa y conocimiento de los parámetros de configuración de Nginx. Esto garantizó una implementación rápida, evitando errores y problemas de no conocer un servicio de balanceo en la nube que no se conocía.
- **PM2:** Gestor de procesos en producción elegido para centralizar el control de las aplicaciones Node.js, tanto Backend como Frontend. Permite realizar reinicios masivos y mantener los servicios en línea frente a caídas inesperadas, agilizando la administración del sistema.
- **NestJS:** Framework utilizado en el desarrollo del Backend, fue seleccionado por su arquitectura fuertemente estructurada y modular. Justifica su uso al facilitar la implementación de APIs REST, la gestión segura de usuarios y roles, y garantizar que el código sea mantenible a medida que el sistema escala.
- **React con TypeScript:** Librería de Frontend indispensable para este sistema audiovisual debido a su naturaleza dinámica. Es vital para reflejar eventos en tiempo real, como votaciones, control de asistencia y turnos de palabra. La integración con TypeScript previene errores mediante tipado estático, robusteciendo el desarrollo a gran escala.
- **TailwindCSS:** Framework de diseño que agiliza la creación de interfaces responsive y uniformes. Su adopción evita la sobrecarga y el desorden de gestionar hojas de estilo CSS tradicionales, permitiendo concentrarse en la lógica central de la aplicación.

Consideración importante

Problema: Inconsistencia de datos por almacenamiento local (Arquitectura Stateful) La raíz del problema radica en el diseño del almacenamiento, los archivos subidos por los usuarios se guardan físicamente en la carpeta local (uploads/) de cada servidor. Dado que Nginx distribuye las peticiones mediante un algoritmo de turnos (Round-Robin), el archivo queda aislado únicamente en la máquina que atendió la subida. Cuando un usuario intenta visualizar ese archivo y Nginx lo redirige al servidor alternativo, el archivo no se encuentra y el sistema arroja un Error 404. Esta separación de datos implica, además, un alto riesgo de pérdida total si la máquina original sufre un apagón o falla crítica.

Intentos de Solución: Sincronización híbrida (lsyncd) y recuperación al arranque (@reboot) Para intentar solucionar este problema sin salir de la arquitectura de discos locales, se probaron distintas estrategias utilizando rsync sobre SSH:

1. **Sincronización en tiempo real:** Se implementó lsyncd para vigilar las carpetas y replicar cambios casi al instante, reduciendo el punto ciego de 10 a 1 segundo y con un bajo consumo de CPU. Esta vigilancia alerta unicamente cuando existe un cambio.
2. **Intentos de recuperación ante apagones:** Para evitar la pérdida de datos cuando un servidor se apaga y se vuelve a encender, se intentó forzar una recuperación automática utilizando la directiva @reboot en el crontab de Linux. La idea era que, al encenderse, la máquina esperara unos segundos a que la red estuviera lista y extrajera mediante rsync todos los archivos faltantes desde la máquina a la que apunta (Maquina original a la replica y viceversa)

Solución a futuro.
Para evitar definitivamente los problemas y errores de sincronización de archivos entre los discos locales, la solución a futuro consistirá en utilizar los servicios de almacenamiento de Google Cloud. De esta manera, los archivos no dependerán de una máquina en específico, permitiendo que todos los servidores guarden y lean la información desde un mismo lugar centralizado y eliminando por completo el riesgo de inconsistencias.

Conclusión

La implementación de esta arquitectura en Google Cloud ha permitido fortalecer un entorno seguro, escalable y resiliente para el sistema audiovisual de CORES. La decisión de segmentar la infraestructura en una red pública y otra privada, sumada al uso de un server-monitor, redujo las posibilidades de ataques externos, blindando los datos y la lógica de negocio contra accesos no autorizados.

Además, el uso del protocolo de respaldos 3-2-1 automatizado hacia Cloud Storage y la utilización de una base de datos Cloud SQL gestionada garantizan no solo la alta disponibilidad del sistema, sino también su capacidad de recuperación ante posibles desastres. En conjunto, las tecnologías seleccionadas y las políticas de firewall aplicadas cumplen con los requisitos exigidos, entregando una plataforma robusta y lista para operar en un entorno de producción exigente.